

Running the μ OS++ xPack Test Harness on the AArch32 and AArch64 Ports

- Running the μ OS++ xPack Test Harness on the AArch32 and AArch64 Ports
 - A step-by-step handbook for the `Work-smp` workspace
 - 1. What you end up with
 - 2. The two test systems, in plain English
 - 3. Prerequisites
 - 4. The target layout
 - 5. Step 1 — Create the workspace and clone the three repositories
 - 6. Step 2 — Create the symlinks
 - 7. Step 3 — Bring the xPack harness into the kernel clone
 - 8. Step 4 — Point the harness at the AArch32 port
 - 9. Step 5 — Keep the C++ standard at 20
 - 10. Step 6 — Create the platform
 - 11. Step 7 — Edit `tests/package.json`
 - 12. Step 8 — Give the kernel a `package.json`
 - 13. Why the semihosting implementation matters
 - 14. Step 9 — Install, prepare, build, test
 - 15. Step 10 — Hardware: board tests with no QEMU (Luckfox Lyra)
 - 16. Doing the same for AArch64
 - 17. Adding a port test to the harness (`smp-pipeline-test`)
 - 18. Troubleshooting — every problem encountered, and its fix
 - 19. Repeating this for another test, board or architecture
 - 20. File reference
 - 21. Glossary

Running the μ OS++ xPack Test Harness on the AArch32 and AArch64 Ports

A step-by-step handbook for the `Work-smp` workspace

This document records, in plain English and in order, every step that was taken to make the **xPack test harness** (the `tests/` folder of `micro-os-plus-iii`) build and run the **SMP kernel** through the **AArch32 and AArch64 ports** on the **Raspberry Pi Zero 2 W**, under QEMU and (later) on real hardware.

It is written so that the same procedure can be repeated for the next test, the next board (rpi3b, Luckfox Lyra), or another architecture, without re-discovering any of it.

1. What you end up with

A workspace at `~/Work-smp/micro-os-plus-iii/` in which:

- the **kernel is the SMP implementation** (`micro-os-plus-iii-smp`), cloned under the name `micro-os-plus-iii.git`;
- the **drivers** live in `micro-os-plus-iii-devices.git`;
- the **AArch32 port** lives in `micro-os-plus-iii-aarch32.git`, with its own per-board `test/` tree (board code and board tests);
- the **xPack harness** lives in `micro-os-plus-iii.git/tests/` and is driven with `xpm`, exactly like the upstream `README-DEVELOPER.md` workflow;
- a single command builds and runs a test:

```
xpm run test --config aarch32-rpi-zero-2w-cmake-gcc-debug
```

and CTest reports a real **Passed / Failed**, because the test exits through the standard ARM semihosting mechanism.

2. The two test systems, in plain English

Before touching anything, it helps to know that there are **two different test systems** involved, and this guide makes them work together.

2.1 The xPack harness (`tests/`)

This is the system documented in `TESTS-XPACK-SYSTEM.md`. It is a *matrix runner*:

- **Test sources** are platform-independent: `tests/sources/<suite>/` — for example `mutex-stress`, `rtos-apis`, `cmsis-os-validator`.
- **Platforms** are folders under `tests/platforms/<name>/` that describe how to build and run those sources on a particular target.
- **Build configurations** in `tests/package.json` name a (platform × toolchain × build type) combination, e.g. `aarch32-rpi-zero-2w-cmake-gcc-debug`.
- The actual build is CMake + Ninja; the actual run is CTest.
- A test passes when its process exits with code 0.

In the original upstream project the harness tests the **plain** kernel. Here we point it at the **SMP kernel, through the AArch32 port**.

2.2 The SMP family and the AArch32 port

- `micro-os-plus-iii-smp` is the RTOS kernel. It exports the same CMake target name as the plain kernel, `micro-os-plus::iii`, but with SMP support.
- `micro-os-plus-iii-devices` holds the drivers (SD, FatFs, DWC2 USB) and the BCM2837 SoC support.

- `micro-os-plus-iii-aarch32` is the **port**: startup, MMU, exception handlers, the SMP scheduler half, and the per-board support. It exports `micro-os-plus::aarch32`, and it **adds the kernel and the devices itself**.

The last point is the single most important rule:

The AArch32 port is the entry point. It adds the kernel and the devices. Never add the kernel separately, or CMake will complain that `micro-os-plus::iii` is defined twice.

3. Prerequisites

- **xpm** (the xPack package manager) and **Node.js ≥ 20** .
 - **arm-none-eabi-gcc 15.2.1** (xPack) and **aarch64-none-elf-gcc 15.2.1** (xPack, only to build the QEMU boot shim), both already in `~/.local/xPacks/@xpack-dev-tools/`.
 - **qemu-arm 9.x** (xPack) providing `qemu-system-aarch64`.
 - The **bare git repositories** of the SMP family, in `/home/dan/Downloads/GIT/`.
 - The **harness** to copy, in `~/Work/micro-os-plus-iii/micro-os-plus-iii.git/tests/`.
-

4. The target layout

Layout (workspace root)

```
~/Work-smp/micro-os-plus-iii/
├─ micro-os-plus-iii.git/           # the SMP kernel (cloned from -smp, renamed)
│   ├─ package.json                # added by hand (see Step 8)
│   ├─ cmake/ include/ port/ src/ test_smpl/
│   └─ tests/                      # the xPack harness (see Step 3)
│       ├─ package.json            # the matrix (edited, Step 7)
│       ├─ cmake/
│       ├─ sources/                # mutex-stress, rtos-apis, cmsis-os-validator
│       └─ platforms/
│           └─ aarch32-rpi-zero-2w/ # the new platform (Step 6)
├─ micro-os-plus-iii-devices.git/   # drivers
├─ micro-os-plus-iii-aarch32.git/   # the port, with test/boards/rpi-zero-2w/
├─ micro-os-plus-iii-smp -> micro-os-plus-iii.git
└─ micro-os-plus-iii-devices -> micro-os-plus-iii-devices.git
```

The two symlinks exist because the port looks for its dependencies as siblings named `micro-os-plus-iii-smp` and `micro-os-plus-iii-devices`, but we cloned them with a `.git` suffix (to mirror the Work layout).

5. Step 1 — Create the workspace and clone the three repositories

Commands (cwd: ~/Work-smp/micro-os-plus-iii)

```
# cwd: ~/Work-smp/micro-os-plus-iii
mkdir -p ~/Work-smp/micro-os-plus-iii
cd ~/Work-smp/micro-os-plus-iii

git clone /home/dan/Downloads/GIT/micro-os-plus-iii-smp.git      micro-os-plus-iii.git
git clone /home/dan/Downloads/GIT/micro-os-plus-iii-devices.git micro-os-plus-iii-devices.git
git clone /home/dan/Downloads/luckfox_lyra/TMP7/micro-os-plus-iii-aarch32 \
    micro-os-plus-iii-aarch32.git
```

Why these sources:

- The kernel comes from the **bare SMP repo**, so it is the SMP implementation.
- The port comes from the **TMP7 working copy** because that is where the newest layout lives: the board code and the board tests have moved under test/ (test/boards/<board>/, test/<board>/). The bare aarch32 repo is behind that refactor.

At this point the kernel is named `micro-os-plus-iii.git`, but it **is** the SMP kernel. Verify:

Commands (cwd: ~/Work-smp/micro-os-plus-iii)

```
# cwd: ~/Work-smp/micro-os-plus-iii
git -C micro-os-plus-iii.git remote -v      # origin = ../micro-os-plus-iii-smp.git
grep -r OS_USE_SMP_SCHEDULER micro-os-plus-iii.git/include | head -1
```

6. Step 2 — Create the symlinks

Commands (cwd: ~/Work-smp/micro-os-plus-iii)

```
# cwd: ~/Work-smp/micro-os-plus-iii
cd ~/Work-smp/micro-os-plus-iii
ln -s micro-os-plus-iii.git      micro-os-plus-iii-smp
ln -s micro-os-plus-iii-devices.git micro-os-plus-iii-devices
```

The port's CMakeLists defaults are:

File: `micro-os-plus-iii-aarch32.git/CMakeLists.txt`

```
# micro-os-plus-iii-aarch32.git/CMakeLists.txt
set (UOS_SMP_DIR      "${uos_siblings}/micro-os-plus-iii-smp"      ...)
set (UOS_DEVICES_DIR "${uos_siblings}/micro-os-plus-iii-devices" ...)
```

The symlinks make those defaults resolve to our clones, so no `-D` flags are needed. Verify:

Commands (cwd: ~/Work-smp/micro-os-plus-iii)

```
# cwd: ~/Work-smp/micro-os-plus-iii
readlink -f micro-os-plus-iii-smp      # ../micro-os-plus-iii.git
readlink -f micro-os-plus-iii-devices  # ../micro-os-plus-iii-devices.git
```

7. Step 3 — Bring the xPack harness into the kernel clone

The harness is tracked in the plain repository. Copy only the tracked files (so no `build/`, `node_modules/` or `xpacks/`):

Commands (cwd: ~/Work-smp/micro-os-plus-iii)

```
# cwd: ~/Work-smp/micro-os-plus-iii
git -C ~/Work/micro-os-plus-iii/micro-os-plus-iii.git archive HEAD tests \
| tar -x -C ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git
```

Result: `micro-os-plus-iii.git/tests/` with `cmake/`, `sources/`, `platforms/`, `package.json`, and so on.

The harness is only the *framework*. It contains no kernel, so copying it from the plain repository does not pull in the plain kernel. What the tests link is decided later, in Step 4, when we point the harness at the port.

8. Step 4 — Point the harness at the AArch32 port

The harness has exactly one line that decides “the library under test”. It is in `tests/cmake/tests-main.cmake`:

File: `micro-os-plus-iii.git/tests/cmake/tests-main.cmake`

```
# micro-os-plus-iii.git/tests/cmake/tests-main.cmake
add_subdirectory (".." "top-bin")
```

We replace it with the port:

File: `micro-os-plus-iii.git/tests/cmake/tests-main.cmake`

```
# micro-os-plus-iii.git/tests/cmake/tests-main.cmake
set (UOS_AAARCH32_DIR "${CMAKE_SOURCE_DIR}/../../micro-os-plus-iii-aarch32.git"
    CACHE PATH "µOS++ III AArch32 port working copy")
if (NOT EXISTS "${UOS_AAARCH32_DIR}/CMakeLists.txt")
    message (FATAL_ERROR "Cannot find the AArch32 port at ${UOS_AAARCH32_DIR}")
endif ()
add_subdirectory ("${UOS_AAARCH32_DIR}" "port-bin")
```

Why this works:

- `tests/` is at `micro-os-plus-iii.git/tests`, so `CMAKE_SOURCE_DIR` is the `tests/` directory, and `../../micro-os-plus-iii-aarch32.git` is the port clone beside the kernel.
- Adding the port adds the SMP kernel (`micro-os-plus::iii`) and the devices, exactly once.
- The port's own `test/` tree is **not** added, because the port is not the top-level project here (`PROJECT_IS_TOP_LEVEL` is false).

Why we edit the generated file and not build-helper. The harness's `tests/cmake/tests-main.cmake`, `common-options.cmake` and the platform `platform-library.cmake` carry a “DO NOT EDIT! Automatically generated from build-helper/templates” banner: they are produced by `build-helper/maintenance-scripts/generate-tests-commons.sh`, which copies the templates into `tests/`.

We deliberately do **not** change `build-helper`:

- it is a **pinned, read-only xPack** (`xpacks/@micro-os-plus/build-helper/`, version 2.17.0) shared by every `μOS++` project — patching it would affect them all and would be lost on the next `xpm install`;
- the change is **specific to this workspace**: it makes the harness select the SMP architecture port by `PLATFORM_NAME`, which is meaningless for the upstream plain-kernel harness.

So `tests-main.cmake` is edited **in place**, and the trade-off is recorded here: if the file is ever regenerated (`bash xpacks/@micro-os-plus/build-helper/maintenance-scripts/generate-tests-commons.sh`), this edit must be re-applied. The clean long-term fixes would be either to upstream a “select the port by platform” hook into the `build-helper` template, or to keep the selection in a project-local file the template does not own.

9. Step 5 — Keep the C++ standard at 20

The harness builds with **C++20**. In `tests/CMakeLists.txt`:

File: `micro-os-plus-iii.git/tests/CMakeLists.txt`

```
# micro-os-plus-iii.git/tests/CMakeLists.txt
set (CMAKE_CXX_STANDARD 20)
```

The port *sources* build cleanly at C++20, and the harness uses C++20 (as upstream `micro-os-plus-iii` does). The port's own `uos_add_app()` was also switched from C++23 to C++20. Both AArch32 and AArch64 pass at C++20.

10. Step 6 — Create the platform

Create `tests/platforms/aarch32-rpi-zero-2w/` with six files. Each is explained below.

10.1 `cmake/definitions.cmake`

Names the platform and guards the toolchain version.

File: micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/cmake/definitions.cmake

```
# micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/cmake/definitions.cmake
set (xpack_platform_compile_definition
    "MICRO_OS_PLUS_PLATFORM_AARCH32_RPI_ZERO_2W")

# Reproducibility guard: refuse anything that is not the pinned 15.2 toolchain.
if (DEFINED CMAKE_C_COMPILER_VERSION
    AND NOT CMAKE_C_COMPILER_VERSION MATCHES "^15\\.2\\.")
    message (FATAL_ERROR
        "This platform must be built with the pinned xPack arm-none-eabi-gcc 15.2, "
        "but found ${CMAKE_C_COMPILER} ${CMAKE_C_COMPILER_VERSION}.")
endif ()
```

10.2 `cmake/dependencies-folders.cmake`

Lists the test sources to add. **Do not list the port here** — it is already added by `tests-main.cmake`.

File: micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/cmake/dependencies-folders.cmake

```
# micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/cmake/dependencies-folders.cmake
set (
    xpack_dependencies_folders
    "${CMAKE_SOURCE_DIR}/sources/mutex-stress"
    "${CMAKE_SOURCE_DIR}/sources/rtos-apis"
    "${CMAKE_SOURCE_DIR}/sources/cmsis-os-validator"
    "${CMAKE_SOURCE_DIR}/xpacks/@xpacks/arm-cmsis-rtos-validator"
)
```

10.3 `cmake/platform-library.cmake`

Creates **two** interface libraries (see §17.2 for why the split is needed):

- `micro-os-plus::platform` — the **base**: include dirs, flags, the linker script, the port, and (AArch32) the kernel's C-library groups. Harness suites and port tests both link it.
- `micro-os-plus::platform-support` — adds `src/platform-support.cpp` (startup hooks + strong `main()`), for the **harness suites only**.

File: micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/cmake/platform-library.cmake

```

# micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/cmake/platform-library.cmake
add_library (platform-aarch32-rpi-zero-2w-interface INTERFACE EXCLUDE_FROM_ALL)

target_include_directories (platform-aarch32-rpi-zero-2w-interface INTERFACE "include")

target_compile_definitions (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE
    "${xpack_platform_compile_definition}"
    OS_NCPU=${UOS_BOARD_NCPU}
    OS_USE_SMP_SCHEDULER=1
    TRACE_SEMIHOST QEMU_BUILD __ARM_EABI__ __ARM_ARCH_7A__
    OS_USE_TRACE_SEMIHOSTING_STDOUT
    _GNU_SOURCE
)

target_compile_options (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE $<$<COMPILE_LANGUAGE:C>:-std=gnu11>
    $<$<COMPILE_LANGUAGE:CXX>:-fno-exceptions>
    $<$<COMPILE_LANGUAGE:CXX>:-fno-rtti>
    $<$<COMPILE_LANGUAGE:CXX>:-fabi-version=0>
)

target_link_options (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE -nostartfiles -Wl,--gc-sections "-T${UOS_BOARD_LINKER_QEMU}"
)

target_link_libraries (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE micro-os-plus::aarch32
    micro-os-plus::iii-newlib-reent
    micro-os-plus::iii-semihosting
)

add_library (micro-os-plus::platform ALIAS platform-aarch32-rpi-zero-2w-interface)

# Support (hooks + main) for the harness suites; port tests link the base only.
add_library (platform-aarch32-rpi-zero-2w-support-interface INTERFACE
  EXCLUDE_FROM_ALL)
target_sources (platform-aarch32-rpi-zero-2w-support-interface
  INTERFACE "src/platform-support.cpp")
target_link_libraries (platform-aarch32-rpi-zero-2w-support-interface
  INTERFACE platform-aarch32-rpi-zero-2w-interface)
add_library (micro-os-plus::platform-support
  ALIAS platform-aarch32-rpi-zero-2w-support-interface)

```

Key points:

- `UOS_BOARD_NCPU` and `UOS_BOARD_LINKER_QEMU` are set and cached by the port's board file (`test/boards/rpi-zero-2w/board.cmake`), so the platform can read them.
- The port's interface already carries the CPU flags (`-mcpu=cortex-a53 -marm -mfloat-abi=hard -mfpu=neon-fp-armv8`) **both at compile and at link time**, and the board defines (`SOC_BCM2837`, `SEMIHOST_TRAP_HLT`, `LED_PIN=29`, ...). The platform adds only the bare-metal extras and the linker script.
- `QEMU_BUILD` makes the port select its QEMU semihosting (SVC + reason-by-value); without it the port would use the hardware trap and QEMU would return 1.
- `micro-os-plus::iii-newlib-reent` and `micro-os-plus::iii-semihosting` are the kernel's optional groups that provide the C library and the exit. They live in the **base** because port tests use `printf` too. See section 13.
- The startup hooks and `main()` are in the **support**, so the port tests (which bring their own strong `main/hooks`) link the base and not the support.

10.4 CMakeLists.txt

Builds the test executables and registers them with CTest. It also builds the AArch64→AArch32 QEMU boot shim.

The important part:

File: micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt

```
# micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt
add_test (
  NAME "${PLATFORM_NAME}-mutex-stress-test"
  COMMAND
    qemu-system-aarch64 --machine raspi3b -smp 4 --nographic --serial none
    --semihosting-config enable=on,target=native
    --kernel "${_shim_img}"
    --device "loader,file=${<TARGET_FILE>}mutex-stress-test>.bin,addr=0x10000"
)
set_tests_properties ("${PLATFORM_NAME}-mutex-stress-test" PROPERTIES
  TIMEOUT 120)
```

and the raw-image step, needed because the shim and the boot card consume a `.bin`, not an ELF:

File: micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt

```
# micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt
add_custom_command (TARGET ${name} POST_BUILD
  COMMAND ${CMAKE_OBJCOPY} -O binary "${<TARGET_FILE>}:${name}>" "${<TARGET_FILE>}:${name}>.bin"
  COMMAND ${CMAKE_SIZE} --format=berkeley "${<TARGET_FILE>}:${name}>"
  VERBATIM)
```

10.5 `include/cmsis-plus/platform.h`

The harness's `os-app-config.h` includes `<cmsis-plus/platform.h>`; the port does not provide one, so the platform does. It also enables the kernel's semihosting syscalls and provides the one CMSIS intrinsic the port lacks.

File: micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/include/cmsis-plus/platform.h

```
// micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/include/cmsis-plus/platform.h
#define OS_USE_SEMIHOSTING_SYSCALLS

#if defined(__arm__) && !defined(__disable_irq)
static inline void __disable_irq (void) { __asm__ volatile ("cpsid i" ::: "memory"); }
static inline void __enable_irq  (void) { __asm__ volatile ("cpsie i" ::: "memory"); }
#endif
```

10.6 `src/platform-support.cpp`

The port's own tests each define their startup hooks and their own `main()`. The harness tests do not, so the platform provides them, mirroring the port's tests:

- `os_startup_initialize_hardware_early()` — empty.
- `os_startup_initialize_hardware()` — initialise the UART, the free store, the exception handlers.
- a **strong** `main()` — set the interrupts stack (the kernel's weak `main` does not, which leaves a 0-byte IRQ stack and hangs the scheduler), create the main thread, start the scheduler.
- the trampoline calls `os_startup_initialize_args()` and then `os_main(argc, argv)`, honouring the harness contract.

The full file:

File: micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/src/platform-support.cpp

```

// micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/src/platform-support.cpp
#include <cmsis-plus/rtos/os.h>
#include <cstdint>
#include <cstdlib>
#include <uart.hpp>
#include <exception_handler.hpp>

extern "C"
{
    extern char __heap_start[], __heap_end[], __fiq_stack_top[], __irq_stack_top[];
    extern void os_startup_create_thread_idle (void);
    extern os::rtos::thread* os_main_thread;

    void os_startup_initialize_hardware_early (void) { }

    void os_startup_initialize_hardware (void)
    {
        uart::uart1.init ();
        os_startup_initialize_free_store (
            __heap_start, static_cast<std::size_t> (__heap_end - __heap_start));
        exception::init ();
    }

    [[noreturn]] static void harness_main_trampoline (void)
    {
        int argc = 0;
        char** argv = nullptr;
        os_startup_initialize_args (&argc, &argv);
        int code = os_main (argc, argv);
        std::exit (code);
    }

    int main (int, char*[])
    {
        using namespace os::rtos;
#ifdef OS_HAS_INTERRUPTS_STACK
        interrupts::stack ()->set (
            reinterpret_cast<thread::stack::element_t*> (__fiq_stack_top),
            __irq_stack_top - __fiq_stack_top);
        interrupts::stack ()->initialize ();
#endif
        scheduler::initialize ();

        static thread::stack::element_t main_stack[8192];
        thread::attributes attr = thread::initializer;
        attr.th_stack_address = main_stack;
        attr.th_stack_size_bytes = sizeof (main_stack);

        static thread main_thread{
            "main", reinterpret_cast<thread::func_t> (harness_main_trampoline),
            nullptr, attr

```



```
// micro-os-plus-iii.git/tests/package.json
"aarch32-rpi-zero-2w-cmake-gcc-debug": {
  "inherit": [ "aarch32-actions", "cmake-actions",
    "aarch32-dependencies", "short-win-paths-properties" ],
  "properties": {
    "buildType": "Debug",
    "platformName": "aarch32-rpi-zero-2w",
    "toolchainFileName": "arm-none-eabi-gcc.cmake",
    "shortConfigurationName": "a32d"
  }
},
"aarch32-rpi-zero-2w-cmake-gcc-release": {
  "inherit": [ "aarch32-rpi-zero-2w-cmake-gcc-debug" ],
  "properties": { "buildType": "MinSizeRel", "shortConfigurationName": "a32r" }
}
```

11.3 An aggregate action

File: micro-os-plus-iii.git/tests/package.json

```
// micro-os-plus-iii.git/tests/package.json
"test-aarch32-rpi-zero-2w-cmake": [
  "xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-debug",
  "xpm run build   --config aarch32-rpi-zero-2w-cmake-gcc-debug",
  "xpm run test    --config aarch32-rpi-zero-2w-cmake-gcc-debug",
  "xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-release",
  "xpm run build   --config aarch32-rpi-zero-2w-cmake-gcc-release",
  "xpm run test    --config aarch32-rpi-zero-2w-cmake-gcc-release"
]
```

11.4 Per-test actions (so the plugin shows each test by name)

The xPack VS Code plugin lists **actions**, not CTest tests, and the inherited `test` action is one per configuration — so every test collapses into a single `<config>-test` entry. Give each test its own named action, on the QEMU platforms and on the hardware platform alike:

File: micro-os-plus-iii.git/tests/package.json

```
// micro-os-plus-iii.git/tests/package.json (inside each *-cmake-gcc-debug config)
"actions": {
  "test-mutex-stress": "{{ properties.commandCMakeReconfigure }} && {{ properties.commandCMakeBuild }} && cd {
  "test-smp-pipeline": "{{ properties.commandCMakeReconfigure }} && {{ properties.commandCMakeBuild }} && cd {
  "test-smp-pro-cons": "{{ properties.commandCMakeReconfigure }} && {{ properties.commandCMakeBuild }} && cd {
}
```

Both `aarch32-rpi-zero-2w-cmake-gcc-debug` and `aarch64-rpi-zero-2w-cmake-gcc-debug` carry the same three (the build folder is resolved per configuration), and the release configs inherit them. Reload the VS Code window to pick them up.

11.5 Repoint the default and CI actions

`install`, `test`, `test-all`, `test-ci` and `install-ci` were changed to use the AArch32 configuration instead of the plain-kernel ones. The old plain configurations (`native`, `qemu-cortex-*`, `nucleo-*`, `raspberrypi-pico`) are left defined but unreferenced; they would fail against the SMP kernel because they expect `micro-os-plus::iii-cortexm / iii-posix-arch`.

12. Step 8 — Give the kernel a `package.json`

The harness prints a greeting by reading `../package.json` relative to `tests/`. The SMP kernel repo has no `package.json`, so add a minimal one at `micro-os-plus-iii.git/package.json`:

File: `micro-os-plus-iii.git/package.json`

```
{
  "name": "@micro-os-plus/micro-os-plus-iii-smp",
  "version": "7.1.0",
  "description": "A source code library with the portable part of μOS++ III, SMP variant",
  "license": "MIT"
}
```

Without it, `cmake` stops with file failed to open for reading: `.../tests/..package.json`.

13. Why the semihosting implementation matters

This is the heart of making CTest work, and it is the part that is easy to get wrong.

13.1 The problem

A CTest test passes only if its process exits with code 0. For a QEMU run, the process exit code comes from the guest's **semihosting** `SYS_EXIT` call.

The AArch32 port has its own semihosting header (`semihosting.hpp`). It used `HLT #0xF000` unconditionally — correct for the **OpenOCD aarch64 debug target** used on real hardware, but QEMU's AArch32 `SYS_EXIT` through that form returns exit code **1**, so CTest reports **Failed** even though the test printed everything and finished. (The trap is now selected by build; see §15.2.)

13.2 The correct implementation

The **original** `micro-os-plus-iii` harness does not use `HLT`. It uses the classic AArch32 semihosting trap **SWI 0x123456** (also written `SVC 0x123456`), implemented in the kernel files:

- `src/libc/newlib/c-newlib-reent.cpp` — the newlib reentrant syscalls (`_write_r`, `_read_r`, `_gettimeofday_r`, ...) which call `__posix_*`;
- `src/semihosting/c-syscalls-semihosting.cpp` — the `__posix_*` layer and the **exit procedure** `os_terminate()` → `report_exception()` → **SWI 0x123456**.

In the **plain** kernel these two files are part of the core. In the **SMP** kernel they were made optional CMake groups:

- `micro-os-plus::iii-newlib-reent`
- `micro-os-plus::iii-semihosting`

They are enabled by defining `OS_USE_SEMIHOSTING_SYSCALLS` (which the harness platforms normally do in their `include/cmsis-plus/platform.h`).

13.3 What we did

- Linked `micro-os-plus::iii-newlib-reent` and `micro-os-plus::iii-semihosting`: the C library (newlib `_write_r`, `_gettimeofday_r`, ... → the `__posix_*` layer) and a working `exit`.
- Defined `OS_USE_SEMIHOSTING_SYSCALLS` in the platform header, plus `_GNU_SOURCE` (for `S_IREAD`) and a `__disable_irq` shim.
- Kept a strong `main()` (for the interrupts stack) that calls `os_main(argc, argv)`.
- The port carries a **strong** `_Exit()` (`src/semihosting-exit.cpp`) that routes `std::exit()` through the port's own `semihosting.hpp`, whose trap is chosen by build. It overrides the kernel's weak `_Exit()`.

So the exit path is:

Exit path

```
os_main returns
-> std::exit(code)
-> _Exit() (strong, the port: src/semihosting-exit.cpp)
-> semihosting::exit_success() / exit_failure()
-> trap by build (QEMU_BUILD: SVC 0x123456, reason by value 0x20026)
-> QEMU exits 0 -> CTest "Passed"
```

The kernel's `iii-semihosting` still supplies `os_terminate()` and the `__posix_*` syscalls; its weak `_Exit()` is shadowed by the port's strong one.

Do **not** link `micro-os-plus::iii-trace-semihosting`: the AArch32 port already defines `os::trace::write` in `src/rtos/os-core.cpp`, and linking the kernel's `trace-semihosting` produces a *multiple definition* link error.

13.4 The linking conditions for CTest (common vs architecture code)

For a C++ test to make CTest report **Passed**, the process must exit with code 0, which on these bare-metal targets means the guest must issue a semihosting `SYS_EXIT` carrying the success reason, in the exact form the host (QEMU or OpenOCD) expects. Getting there depends on the right code being linked **strongly** at two layers.

Common code — the kernel (or the harness standing in for it).

- The C library and the `exit` come from the SMP kernel's *optional* groups:
 - `micro-os-plus::iii-newlib-reent` — the newlib reentrant syscalls (`_write_r`, `_gettimeofday_r`, ...) which call `__posix_*`;

- `micro-os-plus::iii-semihosting` — the `__posix_*` layer and `os_terminate()` → `report_exception()` → SWI 0x123456. On the *plain* kernel these two files are part of the core; on the *SMP* kernel they must be linked explicitly.
- The kernel's `_Exit()` is **weak**; the port's strong `_Exit()` (`src/semihosting-exit.cpp`) shadows it and exits through the port's semihosting. If neither the port's strong `_Exit()` nor the kernel's `os_terminate()` reaches semihosting, the run resets or idles and the test **hangs** until the CTest timeout.
- `OS_USE_SEMIHOSTING_SYSCALLS` must be defined (the `qemu-cortex/nucleo` platforms do it in `include/cmsis-plus/platform.h`), or `c-syscalls-semihosting.cpp` compiles to nothing and `_write/_gettimeofday` stay undefined.
- Supporting macros: `_GNU_SOURCE` (for `S_IREAD`) and, on a port with no CMSIS core, a `_disable_irq` shim.
- A **strong** `main()` that sets the interrupts stack is required — the kernel's weak `main()` does not, and the scheduler hangs. Harness suites get it from `platform-support.cpp`; port tests bring their own.

Architecture-dependent code — the port.

- The kernel's semihosting is **AArch32/Cortex-M only** (it uses `SWI/BKPT` and `r0/r1`). It cannot be linked on AArch64.
- **AArch32**: the port's `include/semihosting.hpp` selects the trap by build variant — `QEMU_BUILD` → `SVC 0x123456` with the reason passed **by value** (QEMU), hardware → `HLT #0xF000` (Pi) or `SVC` (Lyra) with the reason passed **by pointer** (OpenOCD). The port adds a **strong** `_Exit()` (`src/semihosting-exit.cpp`) that overrides the kernel's weak one and exits through this trap.
- **AArch64**: the port provides everything itself — the semihosting (`HLT #0xF000`, two-field { `reason`, `status` } exit), the syscalls (`_write`, `_read`, `_close`, `_lseek`, `_fstat`, `_isatty` in `test/boards/rpi-zero-2w/include/uart.hpp` and `src/handlers.cpp`) and a strong `_Exit()`. The kernel's `iii-semihosting` / `iii-newlib-reent` groups must **not** be linked (AArch32-only); `_gettimeofday` is added by the harness platform.

In one line: link the *strong* semihosting implementation for the target — the kernel's `iii-newlib-reent` + `iii-semihosting` on AArch32, the port's own on AArch64 — and make sure exactly one strong `main()/_Exit()` is present, so `SYS_EXIT` reaches the host with the success reason and the process exits 0.

14. Step 9 — Install, prepare, build, test

The order matters, because the pinned tools live **inside the build folder**.

Commands (cwd: `~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests`)

```
# cwd: ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
cd ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests

# 1. top-level tools (cmake, ninja, build-helper, validator, chan-fatfs)
npm install
xpm install

# 2. the pinned cross tools for this configuration
xpm run install --config aarch32-rpi-zero-2w-cmake-gcc-debug

# 3. configure (this is where the toolchain file is applied)
xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-debug

# 4. build
xpm run build --config aarch32-rpi-zero-2w-cmake-gcc-debug

# 5. run under QEMU via CTest
xpm run test --config aarch32-rpi-zero-2w-cmake-gcc-debug
```

Or all of it at once (debug + release):

Commands (cwd: ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests)

```
# cwd: ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
xpm run test-aarch32-rpi-zero-2w-cmake -C ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/
tests
```

Why `install` must come before `prepare/build`

`xpm install --config X` installs the configuration's tools into `build/X/xpacks/`, and `xpm` prepends `build/X/xpacks/.bin` to `PATH` while it runs `prepare` and `build`. That is how the bare name `arm-none-eabi-gcc` in the toolchain file resolves to the **pinned xPack 15.2.1**.

Consequences:

- If you delete `build/<config>/`, you delete the pinned tools too. Re-run `xpm run install --config <config>` before `prepare`.
- If you run bare `cmake` instead of `xpm run`, the system `arm-none-eabi-gcc` (e.g. 16.2.0) may be found instead. The guard in `definitions.cmake` stops the build with a clear message in that case.

Expected result

Output

```
1/1 Test #1: aarch32-rpi-zero-2w-mutex-stress-test ... Passed    35.68 sec
100% tests passed, 0 tests failed out of 1
```

with output ending in `Interrupts stack: ... bytes used` and `Hasta la Vista!`.

15. Step 10 — Hardware: board tests with no QEMU (Luckfox Lyra)

Some boards have no useful emulator. The Luckfox Lyra is the case: QEMU models none of the RK3506's blocks, so an emulated Lyra would be a generic Cortex-A7 wearing the name, and every test that makes the board worth having — the DWC2 gadget, the SD host, the Cortex-M0 mailbox — would be exactly the part not modelled. The port says so by leaving `UOS_BOARD_LINKER_QEMU` unset; the harness platform then builds **one image per test** (`hwd`) and runs it on silicon through OpenOCD.

15.1 The platform

`tests/platforms/aarch32-luckfox-lyra/`:

- `cmake/definitions.cmake` selects the board **before** the port is added (the port reads `BOARD` to pick its `board.cmake`):

```
set (BOARD "luckfox-lyra" CACHE STRING "" FORCE)
```

- `cmake/platform-library.cmake` defines the **base** (no `QEMU_BUILD`) and, for the harness suites, a **support** interface (§15.4):

```
target_compile_definitions (... INTERFACE
  OS_NCPU=${UOS_BOARD_NCPU} OS_USE_SMP_SCHEDULER=1
  TRACE SEMIHOST HW_BUILD __ARM_EABI__ __ARM_ARCH_7A__ _GNU_SOURCE)
target_link_options (... INTERFACE -nostartfiles -Wl,--gc-sections
  "-T${UOS_BOARD_LINKER_HW}")
```

The board tests link the **base only**: they bring their own strong `main()` and startup hooks, so the support (a second strong `main()`) is a separate interface the harness suites link instead — see §15.4.

- `CMakeLists.txt` builds the port tests and, opt-in, registers their runs. A small `add_hw_test()` helper sets what the port's own `test/CMakeLists.txt` would set for an `hwd` application, so each test is one block:

```

option (ENABLE_HW_TESTS "Register the hardware tests (needs the board)" OFF)

function (add_hw_test _app)
  cmake_parse_arguments (T "" "SECONDS" "SOURCES;INCLUDES;LIBRARIES" ${ARGN})
  add_executable (${_app} ${T_SOURCES})
  target_include_directories (${_app} PRIVATE ${T_INCLUDES})
  set_target_properties (${_app} PROPERTIES OUTPUT_NAME "${_app}-hwd"
    RUNTIME_OUTPUT_DIRECTORY "${CMAKE_CURRENT_BINARY_DIR}/test" SUFFIX "")
  target_link_libraries (${_app} PRIVATE micro-os-plus::iii
    micro-os-plus::platform ${T_LIBRARIES})

  if (ENABLE_HW_TESTS)
    set (_secs "${T_SECONDS}")
    if (_secs STREQUAL "")
      set (_secs 600)          # run budget passed to run-hw.sh
    endif ()
    add_test (NAME "${PLATFORM_NAME}-${_app}"
      COMMAND bash "${_port_hw}" ${_app} ${_secs})
    math (EXPR _timeout "${_secs} + 600")
    set_tests_properties ("${PLATFORM_NAME}-${_app}" PROPERTIES
      ENVIRONMENT "BUILD=${CMAKE_CURRENT_BINARY_DIR}" TIMEOUT ${_timeout})
    endif ()
endfunction ()

add_hw_test (smp_test4
  SOURCES "${_port_test_dir}/smp_test4/main.cpp"
    "${_port_test_dir}/src/test-smp-boot.cpp"
  INCLUDES "${_port_test_dir}/include" "${_port_test_dir}/smp_test4")

add_hw_test (smp-mat-sdcard-test
  SOURCES "${_port_test_dir}/smp-mat-sdcard-test/main.cpp"
    "${_port_test_dir}/src/test-smp-boot.cpp"
  INCLUDES "${_port_test_dir}/include" "${_port_test_dir}/smp-mat-sdcard-test"
  LIBRARIES micro-os-plus::devices-rk3506
  SECONDS 1800)          # the SD solver runs for many minutes

add_hw_test (smp_test5
  SOURCES "${_port_test_dir}/smp_test5/main.cpp" ${_fatfs}
  INCLUDES "${_fatfs_dir}" "${_port_test_dir}/smp_test5"
  LIBRARIES micro-os-plus::devices-rk3506
  SECONDS 900)

```

hw.sh runs `run-hw.sh "$BUILD/test" <app>`, so each image must be `<binary>/test/<app>-hwd` — hence the output name, runtime dir and cleared suffix. `SECONDS` is the run budget the runner gets; the CTest `TIMEOUT` is that plus a margin, so CTest never kills a run the runner would still allow.

- `package.json` gets one **named action per test** (the xPack VS Code plugin lists actions, not CTest tests, so this is what gives each test its own name):

```

"actions": {
  "test":
    "cd {{ properties.buildFolderRelativePath }} && ctest -V -LE hw",
  "test-mutex-stress":
    "{{ properties.commandCMakeReconfigure }} &&
    {{ properties.commandCMakeBuild }} && cd {{ properties.buildFolderRelativePath }}
    && ctest -V -R mutex-stress",
  "test-smp_test4":
    "{{ properties.commandCMakeReconfigure }} &&
    {{ properties.commandCMakeBuild }} && cd {{ properties.buildFolderRelativePath }}
    && ctest -V -R smp_test4",
  "test-smp-mat-sdcard-test":
    "{{ properties.commandCMakeReconfigure }} &&
    {{ properties.commandCMakeBuild }} && cd {{ properties.buildFolderRelativePath }}
    && ctest -V -R smp-mat-sdcard-test",
  "test-smp_test5":
    "{{ properties.commandCMakeReconfigure }} &&
    {{ properties.commandCMakeBuild }} && cd {{ properties.buildFolderRelativePath }}
    && ctest -V -R smp_test5"
}

```

Every hardware test carries LABELS `hw` in `add_hw_test`, and the generic test action runs `ctest -LE hw`: on the Lyra that selects **nothing** (all four tests are hardware tests), so a stray `test` cannot start a board run — use the named `test-*` actions, one per power cycle. The QEMU platforms keep the inherited `test` action unchanged.

15.2 The strong semihosting paradigm

On hardware there is **no process status** to compare: OpenOCD is a debugger, not a parent. The verdict is therefore a line the runner greps, and the exit is what *stops* the run. The port's `include/semihosting.hpp` chooses the trap by build (see §13):

Build	Trap	SYS_EXIT reason
QEMU_BUILD	<code>svc 0x123456</code>	value in <code>r1</code>
SEMIHOST_TRAP_HLT (Pi)	<code>hlt #0xF000</code>	pointer
neither (Lyra, cortex_a)	<code>svc 0x123456</code>	pointer

and adds the verdict-and-stop the hardware tests end with:

File: micro-os-plus-iii-aarch32.git/include/semihosting.hpp

```

// micro-os-plus-iii-aarch32.git/include/semihosting.hpp
[[noreturn]] inline void
report_result (bool pass) noexcept
{
  write_str (pass ? "\nRESULT: PASS\n" : "\nRESULT: FAIL\n"); // the verdict
  pass ? exit_success () : exit_failure ();                    // the stop
}

```

The strong `_Exit()` (`src/semihosting-exit.cpp`) routes `std::exit()` through the same path, so a test that returns from `os_main()` and one that calls `report_result()` both end deterministically. Every failure path in `smp_test5` calls `report_result (false)`.

`hlt #0xF000` has no encoding on ARMv7-A: the word is an Undefined Instruction, so on the Lyra the first character would fault and, if the fault handler also prints, the board

loops printing fault dumps. HLT is for the Pi (whose OpenOCD target is aarch64); the Lyra uses the ARMv7-A Angel SVC.

15.3 Build and run

Commands (cwd: ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests)

```
# cwd: ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
cd ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
xpm run install --config aarch32-luckfox-lyra-cmake-gcc-debug
xpm run prepare --config aarch32-luckfox-lyra-cmake-gcc-debug
xpm run build --config aarch32-luckfox-lyra-cmake-gcc-debug

# register the runs (board attached; one power cycle per test)
PATH="$PWD/build/aarch32-luckfox-lyra-cmake-gcc-debug/xpacks/.bin:$PATH" \
  cmake -DENABLE_HW_TESTS=ON build/aarch32-luckfox-lyra-cmake-gcc-debug
ctest --test-dir build/aarch32-luckfox-lyra-cmake-gcc-debug -N

# run one
ctest --test-dir build/aarch32-luckfox-lyra-cmake-gcc-debug -V -R smp_test4

# or by hand
BUILD=$PWD/build/aarch32-luckfox-lyra-cmake-gcc-debug/platform-bin \
  ../../micro-os-plus-iii-aarch32.git/test/boards/luckfox-lyra/hw.sh smp_test4
```

The registered tests, each also a named plugin action (test-<app>):

Test	Needs	Budget	Verdict
smp_test4	shared test-smp-boot.cpp	600 s	hw_result::ok() → semihosting
smp-mat-sdcard-test	+ devices (SD/FatFs), existing FAT32	1800 s	hw_result::ok()
smp_test5	self-contained + C++ FatFs + SD host	900 s	report_result()
mutex-stress-test	harness suite + board platform-support	600 s	support trampoline

Expected: the banner, the test's log, RESULT: PASS, then [hw] <app> PASSED.

Hardware bring-up notes

Three things were needed to make the OpenOCD run usable, all read at run time (no rebuild):

- **SWD clock.** test/boards/luckfox-lyra/hw.sh (U0S_HW_ADAPTER_KHZ) and openocd.cfg (adapter speed) carry it; it was raised from 1000 to 4000 kHz for throughput. If you see Invalid ACK / DEBUG LINK LOST, drop it back (U0S_HW_ADAPTER_KHZ=2000).
- **Console throughput.** Semihosting costs one debug trap per call, so the port's uart.hpp mirrors the console **line-buffered**: putc/operator<< accumulate and emit one SYS_WRITE0 per line, not one SYS_WRITEC per character. A partial line needs uart::uart1.flush().

- **OpenOCD log noise.** `openocd.cfg`'s `examine_secondaries` now runs once, silently, and wraps the examination in `log_output /dev/null ... log_output default`, so the `Info : [<target>] hardware has N breakpoints` lines no longer interleave with the target's semihosting.

Requirements and rules:

- An OpenOCD-compatible probe (the Lyra's config is `test/boards/luckfox-lyra/openocd.cfg`).
- **One test per power cycle.** The secondaries are released once, by clearing their CRU reset bits, and a second load cannot un-release them; `run-hw.sh` refuses to run a suite for exactly this reason.
- Keep your own terminal on the UART; the runner does not open it. The verdict is read from the semihosted console in OpenOCD's log.
- The Pi still uses `HLT #0xF000` (its OpenOCD target is `aarch64`); the Lyra uses the ARMv7-A Angel `SVC 0x123456`. See the table above.

15.4 A harness suite on the board

The tests above are the port's own; they bring their own `main()` and hooks. A **harness suite** (`tests/sources/`, e.g. `mutex-stress`) does not, so it needs a board `platform-support` — the hardware variant of what the QEMU platforms already provide:

- `src/platform-support.cpp`: the `os_startup_initialize_hardware*()` hooks, a strong `main()` (interrupts stack, scheduler), and a trampoline that calls `os_main(0, nullptr)`, prints `RESULT: PASS|FAIL` on the console (the hardware verdict), and `std::exit`s through the port's strong `_Exit()`.
- `CMake/platform-library.cmake`: a `platform-support` interface that adds that file and links the kernel's `lib-newlib-reent` + `lib-semihosting` (the suites use `printf/gettimeofday`). The base also needs `_GNU_SOURCE` (the syscall layer uses `S_IREAD`), and `platform.h` defines `OS_USE_SEMIHOSTING_SYSCALLS`.
- the suite target links `micro-os-plus::common-options` + `test::mutex-stress`
 - the base + the support; `sources/mutex-stress` goes into `dependencies-folders.cmake`; the `run` and the `test-mutex-stress` action are registered like any other.
- **Open the semihosting fds.** The suites print through the C library (`printf`), whose `_write` looks the fd up in the semihosting table (`__posix_write` → `__semihosting_findslot`). That table is filled by `initialise_monitor_handles()`, which `os_startup_initialize_args()` calls at its end. The QEMU support calls it; a hardware support that passes `os_main(0, nullptr)` does **not**, so the table stays empty and **every printf is discarded** (EBADF) — while the kernel's own unbuffered trace (the `./!` tick markers) still shows, which makes it look like a buffering problem when it is not. The hardware trampoline must call `os_startup_initialize_args(&argc, &argv)` before `os_main`, and `setvbuf (stdout, nullptr, _IOLBF, 0)` — **after** the free store, since `setvbuf` allocates the stdout buffer — to line-buffer the output.

The suite source is unchanged — it returns a code from `os_main()`, and the support trampoline turns that into the `RESULT:` line, so the same source runs under QEMU (exit code) and on the board (log line). On the Lyra this is `aarch32-luckfox-lyra-mutex-stress-test`.

16. Doing the same for AArch64

Everything above applies, with a handful of differences. This section lists them and the exact extra files, so the AArch64 port can be set up by analogy.

16.1 Clone and select

Commands (cwd: ~/Work-smp/micro-os-plus-iii)

```
# cwd: ~/Work-smp/micro-os-plus-iii
cd ~/Work-smp/micro-os-plus-iii
git clone /home/dan/Downloads/GIT/micro-os-plus-iii-aarch64.git micro-os-plus-iii-aarch64.git
```

No symlink is needed for the port itself (the harness finds it through `UOS_AARCH64_DIR`), and the port finds the kernel and devices through the existing `micro-os-plus-iii-smp` / `micro-os-plus-iii-devices` symlinks.

`tests-main.cmake` must choose the port from the platform name:

File: `micro-os-plus-iii.git/tests/cmake/tests-main.cmake`

```
# micro-os-plus-iii.git/tests/cmake/tests-main.cmake
set (UOS_AARCH32_DIR "${CMAKE_SOURCE_DIR}/../../micro-os-plus-iii-aarch32.git" CACHE PATH "")
set (UOS_AARCH64_DIR "${CMAKE_SOURCE_DIR}/../../micro-os-plus-iii-aarch64.git" CACHE PATH "")
if (PLATFORM_NAME MATCHES "^aarch32")
    add_subdirectory ("${UOS_AARCH32_DIR}" "port-bin")
elseif (PLATFORM_NAME MATCHES "^aarch64")
    add_subdirectory ("${UOS_AARCH64_DIR}" "port-bin")
else ()
    add_subdirectory (".." "top-bin") # plain kernel
endif ()
```

16.2 The AArch64 platform

Create `tests/platforms/aarch64-rpi-zero-2w/` with the same six files as for AArch32. The differences:

- **Toolchain:** `aarch64-none-elf-gcc.cmake`; the guard checks 15.2.
- **CPU flags** come from the port (`-mcpu=cortex-a53`, AArch64); no `-marm`.
- **Defines:** `__ARM_ARCH_8A__` instead of `__ARM_ARCH_7A__`.
- **No boot shim:** QEMU's `raspi3b` starts the cores in AArch64, so the test image is handed to `-kernel` directly:

```
add_test (NAME "${PLATFORM_NAME}-mutex-stress-test" COMMAND
    qemu-system-aarch64 --machine raspi3b -smp 4 --nographic --serial none
    --semihosting-config enable=on,target=native
    --kernel "$<TARGET_FILE:mutex-stress-test>.bin")
```

- **Strong semihosting:** the AArch64 port's `semihosting.hpp` uses `HLT #0xF000` with a **two-field** `{ reason, status }` exit block, which QEMU maps to the process exit code. The platform therefore provides a strong `_Exit()` calling `semihosting::exit_success() / exit_failure()`. The kernel's AArch32-only `iii-semihosting` groups are **not** linked.
- **The port already provides the other syscalls:** `_write`, `_read`, `_close`, `_lseek`, `_fstat` and `_isatty` are defined in the port's `test/boards/rpi-zero-2w/include/uart.hpp` and `src/handlers.cpp`. So `platform-support.cpp` adds only `_gettimeofday` and `_Exit`. Defining the others would be a *multiple definition* link error.

16.3 package.json

Add the AArch64 analogues of the AArch32 blocks:

File: [micro-os-plus-iii.git/tests/package.json](#)

```
// micro-os-plus-iii.git/tests/package.json
"aarch64-actions": { "hidden": true, "actions": {
  "install": [ "xpm install --config {{ configuration.name }}" ] } },
"aarch64-dependencies": { "hidden": true, "devDependencies": {
  "@xpack-dev-tools/aarch64-none-elf-gcc": "15.2.1-1.1.1",
  "@xpack-dev-tools/qemu-arm": "9.2.4-1.1" } }
```

plus the two configurations (aarch64-rpi-zero-2w-cmake-gcc-debug and -release, toolchain aarch64-none-elf-gcc.cmake) and the test-aarch64-rpi-zero-2w-cmake aggregate action.

16.4 Build and run

Exactly as for AArch32:

Commands (cwd: ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests)

```
# cwd: ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
cd ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
xpm run install --config aarch64-rpi-zero-2w-cmake-gcc-debug
xpm run prepare --config aarch64-rpi-zero-2w-cmake-gcc-debug
xpm run build --config aarch64-rpi-zero-2w-cmake-gcc-debug
xpm run test --config aarch64-rpi-zero-2w-cmake-gcc-debug
# or both debug and release:
xpm run test-aarch64-rpi-zero-2w-cmake
```

Result (C++20):

Output

```
1/1 Test #1: aarch64-rpi-zero-2w-mutex-stress-test ... Passed 35.6 sec
100% tests passed, 0 tests failed out of 1
```

16.5 Why AArch64 needs no `SWI` groups

On AArch32 the port's `HLT` exit returned QEMU exit code 1 (single-field block), so we linked the kernel's `SWI 0x123456` groups. On AArch64 the port's `HLT` uses the two-field block QEMU expects, so the port's own semihosting is already correct and no kernel group is needed.

17. Adding a port test to the harness (`smp-pipeline-test`)

So far the harness has built one of its own suites (`mutex-stress`). The port also ships its **own** tests under `test/<board>/`, and they can be run through the same CTest system. This chapter adds `smp-pipeline-test` and explains the one structural change it forces.

17.1 Port test vs harness suite

	harness suite	port test
lives in	<code>tests/sources/<name>/</code>	<code><port>/test/<board>/<name>/</code>
provides	only the test logic	its own <code>main()</code> and startup hooks
links	<code>micro-os-plus::platform</code> + <code>platform-support</code>	<code>micro-os-plus::platform</code> only
verdict	returns from <code>os_main</code> (exit code)	prints <code>RESULT:</code> , ends via <code>hw_result</code>

The port tests are the ones that have always run on the Pi; the harness merely gives them a CTest face.

17.2 The platform split (do this once)

A port test defines its own `main()` and `os_startup_initialize_hardware*()`. The harness's `platform-support` also defines them. If both are **strong**, the link fails; if both are **weak**, the linker may pick the kernel's weak `main` (which does *not* set the interrupts stack) and the test hangs. So the platform is split in two:

- `micro-os-plus::platform` — include dirs, flags, the linker script, the port, and (AArch32) the kernel's `iii-newlib-reent` + `iii-semihosting` C library. **Port tests link this.**
- `micro-os-plus::platform-support` — `src/platform-support.cpp` (startup hooks
 - strong `main()`, plus AArch64's `_Exit/_gettimeofday`). **Harness suites link base + support.**

Both `platform-library.cmake` files already implement this split.

17.3 Make the port's QEMU semihosting exit 0

The port test ends through `hw_result::ok()` → the port's semihosting. Under QEMU that must be the classic trap with the reason passed **by value**, so the harness platform defines `QEMU_BUILD`, and the port's `semihosting.hpp` selects the QEMU form (the TMP7 change). If the port clone predates that change, `sync include/semihosting.hpp` first.

17.4 Add the executable

In `tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt` (AArch64 analogous):

File: micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt

```
# micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt
option (ENABLE_SMP_PIPELINE_TEST "Build the port's smp-pipeline-test" ON)
if (ENABLE_SMP_PIPELINE_TEST)
    set (_port_test_dir "${UOS_AARCH32_DIR}/test/rpi-zero-2w")
    add_executable (smp-pipeline-test
        "${_port_test_dir}/smp-pipeline-test/main.cpp"
        "${_port_test_dir}/src/test-smp-boot.cpp")
    target_include_directories (smp-pipeline-test PRIVATE "${_port_test_dir}/include")
    add_custom_command (TARGET smp-pipeline-test POST_BUILD
        COMMAND ${CMAKE_OBJCOPY} -O binary "$<TARGET_FILE:smp-pipeline-test>"
            "$<TARGET_FILE:smp-pipeline-test>.bin"
        VERBATIM)
    target_link_libraries (smp-pipeline-test PRIVATE
        micro-os-plus::common-options micro-os-plus::iii
        micro-os-plus::platform          # base only, NOT platform-support
        micro-os-plus::devices)          # the port test reaches the SD card
    # ... SD image + add_test, below
endif ()
```

Key points:

- Link the **base** `micro-os-plus::platform`, never `platform-support`.
- Link `micro-os-plus::devices` only if the test is in the board's `BOARD_TEST_NEED_DEVICES` list (`sd_test` `smp-mat-sdcard-test` `smp-num-test` `smp-pipeline-test` `usb_test`).
- Include the port test's shared `include/` and compile its `src/test-smp-boot.cpp`.
- The test's own strong `main/hooks` come from its `main.cpp` and override the harness's (which are not linked here).

17.5 The SD image and the CTest command

`smp-pipeline-test` formats a blank SD card with `fatfs`, so the QEMU run needs an SD drive:

File: micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt

```
# micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt
set (_pipe_disk "${CMAKE_CURRENT_BINARY_DIR}/smp-pipeline-test.disk.img")
add_custom_command (OUTPUT "${_pipe_disk}"
    COMMAND truncate -s 4G "${_pipe_disk}" VERBATIM)
add_custom_target (smp-pipeline-test-disk ALL DEPENDS "${_pipe_disk}")

add_test (NAME "${PLATFORM_NAME}-smp-pipeline-test" COMMAND
    qemu-system-aarch64 --machine raspi3b -smp 4 --nographic --serial none
    --semihosting-config enable=on,target=native
    --drive "file=${_pipe_disk},if=sd,format=raw"
    --kernel "${_shim_img}" # AArch32; AArch64 uses the .bin directly
    --device "loader,file=${TARGET_FILE:smp-pipeline-test}.bin,addr=0x10000")
set_tests_properties ("${PLATFORM_NAME}-smp-pipeline-test" PROPERTIES
    TIMEOUT 1200)
```

Prefix every CTest test name with `${PLATFORM_NAME}`. A CTest test name is what every test runner shows as the leaf label; the *executable* name can stay short. Because `add_test(NAME "mutex-stress-test")` is identical in the aarch32 and aarch64 platforms, a runner (VS Code's Test Explorer, the xPack actions view, a flat `ctest -N`) cannot tell them apart. Prefixing gives `aarch32-rpi-zero-2w-mutex-stress-test` and `aarch64-rpi-zero-2w-mutex-stress-test`. `ctest -R <name>` still matches, because it is a substring match.

Do the same for the CMake project name. `tests/CMakeLists.txt` uses `project(micro-os-plus-micro-os-plus-iii-${PLATFORM_NAME}-tests)`, so a tool that groups tests by the CMake project shows two distinct suites (`...-aarch32-rpi-zero-2w-tests` and `...-aarch64-rpi-zero-2w-tests`) instead of one. Without it, both configurations report the same project name.

17.6 Build and run

Commands (cwd: `~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests`)

```
# cwd: ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
cd ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
xpm run build --config aarch32-rpi-zero-2w-cmake-gcc-debug
# all tests, or just the new one:
PATH="$PWD/build/aarch32-rpi-zero-2w-cmake-gcc-debug/xpacks/.bin:$PATH" \
    ctest --test-dir build/aarch32-rpi-zero-2w-cmake-gcc-debug \
        -R smp-pipeline-test -V
```

Result:

Output

```
1/3 Test #1: aarch32-rpi-zero-2w-mutex-stress-test ... Passed    35.8 sec
2/3 Test #2: aarch32-rpi-zero-2w-smp-pipeline-test ... Passed    33.9 sec
3/3 Test #3: aarch32-rpi-zero-2w-smp-pro-cons-test ... Passed     4.8 sec
100% tests passed, 0 tests failed out of 3
```

17.7 Adding the other port tests

The same recipe applies to `sd_test`, `smp-num-test`, `smp-mat-sdcard-test`, `smp-mat-test`, `smp-pro-cons-test`, `smp_test0..4`, and `usb_test`. The only variables are: whether the test needs `micro-os-plus::devices` (the `BOARD_TEST_NEED_DEVICES` list) and whether it needs an SD drive. `usb_test` additionally needs a host-side driver while it runs, so it is not suitable for an unattended CTest run.

Checklist:

- ☐ the port's `semihosting.hpp` has the `QEMU_BUILD` path (or sync it)
- ☐ the platform is split into base + support (done)
- ☐ executable from the port test source + `src/test-smp-boot.cpp`
- ☐ include the port test's `include/`
- ☐ link `micro-os-plus::platform` (base) + `devices` if needed
- ☐ `.bin` step
- ☐ SD image + `--drive` if the test touches the card
- ☐ `add_test` with a generous `TIMEOUT`
- ☐ `ctest -R <name>` passes

18. Troubleshooting — every problem encountered, and its fix

Symptom	Cause	Fix
file failed to open ... tests/.../package.json	the SMP kernel has no <code>package.json</code>	add a minimal one at the kernel root (Step 8)
CMake error: alias <code>micro-os-plus::iii</code> already exists	the kernel was added twice	only add the port; never add the kernel separately
CMake error: source dir already used	the port re-adds the kernel you also added	add only the port
Configure used <code>/usr/bin/cc / arm-none-eabi-gcc 16.2.0</code>	build run without <code>prepare</code> , or build dir deleted	run <code>install</code> then <code>prepare</code> then <code>build</code> ; the guard enforces 15.2
undefined reference to <code>os_startup_initialize_hardware_early/hardware</code>	the harness test does not provide the startup hooks	<code>platform-support.cpp</code> provides them
		link <code>micro-os-plus::iii-newlib-</code>

Symptom	Cause	Fix
undefined reference to <code>_write/_read/_gettimeofday/...</code>	the SMP kernel keeps the C library in optional groups	<code>reent + micro-os-plus::iii-semihosting</code>
undefined reference to <code>os_startup_initialize_args</code>	<code>OS_USE_SEMIHOSTING_SYSCALLS</code> not defined	define it in the platform header
' <code>S_IREAD</code> ' was not declared	GNU extension not enabled	add <code>_GNU_SOURCE</code>
' <code>__disable_irq</code> ' was not declared	the port has no CMSIS core headers	define it in the platform header
multiple definition of <code>os::trace::write</code>	linked both the port's trace and <code>iii-trace-semihosting</code>	do not link <code>iii-trace-semihosting</code>
Test prints everything then <code>***Failed</code> , QEMU exit 1	exit via the port's HLT semihosting	use the kernel's SWI <code>0x123456</code> exit (section 13)
Test hangs after <code>scheduler::start()</code> , Interrupts stack size: 0 bytes	kernel weak main used, interrupts stack not set	provide the strong <code>main()</code> (Step 10.6)
no such file ... <code>xpacks/.bin/arm-none-eabi-gcc</code> after a clean	build dir (with <code>xpacks</code>) was deleted	re-run <code>xpm run install --config <config></code>
guard still reports the wrong compiler after a failed configure	the build dir cached the compiler before the guard fired	delete <code>build/<config>/</code> , then <code>install → prepare → build</code>
multiple definition of <code>_write/_read/_close/_lseek/_fstat/_isatty</code> (AArch64)	the port already defines them (<code>uart.hpp</code> , <code>handlers.cpp</code>)	do not define them in <code>platform-support.cpp</code> ; add only <code>_gettimeofday</code> and <code>_Exit</code>
AArch64 <code>-mcpu/float</code> link error	AArch64 has no <code>-marm/-mfloat-abi</code> ; those are AArch32-only	let the port carry the AArch64 flags

19. Repeating this for another test, board or architecture

17.1 Add another test suite

1. Copy the suite to `tests/sources/<name>/`.
2. Add `"${CMAKE_SOURCE_DIR}/sources/<name>"` to the platform's `dependencies-folders.cmake`.
3. Add an `if (ENABLE_<NAME>_TEST)` block to the platform `CMakeLists.txt` that links `test::<name>` and registers `add_test`.
4. If the suite needs extra defines or libraries (for example `chan-fatfs`), add them in that block.

17.2 Add another board

The AArch32 port already knows `zero2w`, `rpi3b` and `luckfox-lyra` through `test/boards/<board>/board.cmake`. Add a platform per board (or parameterise the platform with `-D BOARD=`), and a matching build configuration in `package.json`.

17.3 Add the AArch64 port

Same shape as AArch32 (see §16 for the full walk-through): clone `micro-os-plus-iii-aarch64.git`, add a platform that links `micro-os-plus::aarch64`, uses `aarch64-none-elf-gcc.cmake`, and runs QEMU `raspi3b` directly (no shim). The AArch64 port provides its **own** semihosting (HLT `#0xF000`, two-field exit) and syscalls, so the kernel's AArch32-only `iii-semihosting` / `iii-newlib-reent` groups must **not** be linked.

17.4 Checklist

- ☐ kernel cloned as `micro-os-plus-iii.git`, from `-smp`
 - ☐ symlinks `micro-os-plus-iii-smp`, `micro-os-plus-iii-devices`
 - ☐ harness `tests/` copied
 - ☐ `tests-main.cmake` adds the port, not `..`
 - ☐ `CMAKE_CXX_STANDARD 20`
 - ☐ platform files: `definitions`, `dependencies-folders`, `platform-library`, `CMakeLists`, `platform.h`, `platform-support.cpp`
 - ☐ `package.json`: hidden blocks, configs, aggregate, repointed defaults
 - ☐ kernel `package.json`
 - ☐ toolchain guard
 - ☐ `install` → `prepare` → `build` → `test` passes
-

20. File reference

Concern	Path (relative to <code>~/Work-smp/micro-os-plus-iii/</code>)
Kernel (SMP)	<code>micro-os-plus-iii.git/</code>
Kernel <code>package.json</code>	<code>micro-os-plus-iii.git/package.json</code>
Harness	<code>micro-os-plus-iii.git/tests/</code>
Harness entry edit	<code>micro-os-plus-iii.git/tests/cmake/tests-main.cmake</code>
Harness standard	<code>micro-os-plus-iii.git/tests/CMakeLists.txt</code>
Harness matrix	<code>micro-os-plus-iii.git/tests/package.json</code>
New platform (AArch32)	<code>micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/</code>
New platform (AArch64)	

Concern	Path (relative to <code>~/Work-smp/micro-os-plus-iii/</code>)
	<code>micro-os-plus-iii.git/tests/platforms/aarch64-rpi-zero-2w/</code>
Platform library (base + support)	<code>.../cmake/platform-library.cmake</code> (<code>micro-os-plus::platform</code> , <code>micro-os-plus::platform-support</code>)
Platform definitions/guard	<code>.../cmake/definitions.cmake</code>
Platform dependencies	<code>.../cmake/dependencies-folders.cmake</code>
Platform executables/tests	<code>.../CMakeLists.txt</code>
Platform header	<code>.../include/cmsis-plus/platform.h</code>
Platform support source	<code>.../src/platform-support.cpp</code> (hooks/main; harness suites)
Port test (smp-pipeline)	<code>micro-os-plus-iii-aarch32.git/test/rpi-zero-2w/smp-pipeline-test/</code>
Devices	<code>micro-os-plus-iii-devices.git/</code>
AArch32 port	<code>micro-os-plus-iii-aarch32.git/</code>
AArch64 port	<code>micro-os-plus-iii-aarch64.git/</code>
Board facts	<code>micro-os-plus-iii-aarch32.git/test/boards/rpi-zero-2w/board.cmake</code>
Hardware runner	<code>micro-os-plus-iii-aarch32.git/test/hw.sh</code>
Build folder	<code>micro-os-plus-iii.git/tests/build/aarch32-rpi-zero-2w-cmake-gcc-debug/</code>

21. Glossary

Term	Meaning
xpm	The xPack package manager CLI.
xPack	An npm package with an "xpack" section; used for tools and source libraries.
build configuration	A named (platform, toolchain, build type) tuple in <code>package.json</code> .
action	A named command run with <code>xpm run</code> .
hidden block	A configuration fragment meant only for <code>inherit</code> .
port	The architecture-specific project (<code>micro-os-plus-iii-aarch32</code>) that adds the kernel and devices and exports <code>micro-os-plus::aarch32</code> .
semihosting	The ARM mechanism by which the target asks the host (QEMU or a debugger) to do I/O and to exit.
SWI/SVC 0x123456	The classic AArch32 semihosting trap.
HLT #0xF000	The AArch64 (and AArch32-under-aarch64-debugger) semihosting trap.

Term	Meaning
<code>_Exit</code>	The libc function <code>exit()</code> ends with; the kernel defines it weak, so a platform can replace it.
<code>os_terminate</code>	The kernel's semihosting exit procedure (weak), supplied when <code>iii-semihosting</code> is linked; on the AArch32 platforms the port's strong <code>_Exit()</code> shadows the kernel's weak one.
interrupts stack	A separate stack used while handling interrupts; the AArch32 port must have it set or the scheduler hangs.
shim	A 20-line AArch64 stub that drops QEMU's Cortex-A53 to AArch32 and jumps to the image at 0x10000.
CTest	CMake's test runner; it checks the process exit code.

Companion to `TESTS-XPACK-SYSTEM.md` and `AARCH32-RPI-ZERO-2W-TESTS.md`. Verified against the master/working branches of the `-smp`, `-devices`, `-aarch32` and `-aarch64` repositories and the `tests/` folder of `micro-os-plus-iii`.